🏛 GLA UNIVERSITY, MATHURA — DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

HPC Masterclass | Day 4 Pre-Assessment: Foundations of Profiling, Microarchitecture & Distributed Systems

Target Audience: 7th Semester B.Tech CSE (HPC Elective) | **Total Questions**: 50 | **Duration**: 60 Minutes

📋 Answer Key & Forensic Solution Guide (Instructor Copy)

Q#	Correct Answer	Cognitive Bloom Level	Core Concept Focus
01	B	`Understand`	What is the primary operational difference between...
02	B	`Analyze`	What is the primary drawback of using source-code ...
03	B	`Knowledge`	What does the metric 'Instructions Per Cycle' (IPC...
04	B	`Understand`	According to Amdahl's Law, what limits the maximum...
05	B	`Knowledge`	In software profiling, what is a performance 'Hots...
06	B	`Understand`	In a profiler call tree, what is the difference be...
07	B	`Analyze`	Why is statistical sampling profiling considered s...
08	B	`Understand`	What is the difference between 'CPU Time' and 'Wal...
09	B	`Analyze`	What problem arises when profiling multi-threaded ...
10	B	`Knowledge`	Why must an executable be compiled with debug symb...
11	B	`Understand`	In modern superscalar processors, what is the role...
12	B	`Understand`	What happens when a CPU branch predictor incorrect...
13	B	`Understand`	What is the purpose of the 'Reorder Buffer' (ROB) ...
14	B	`Analyze`	What is 'Register Renaming' in superscalar process...
15	B	`Knowledge`	What does a Fused Multiply-Add (FMA) hardware inst...
16	B	`Understand`	What is 'Simultaneous Multi-Threading' (SMT), know...
17	B	`Understand`	What is 'Store-to-Load Forwarding' in a CPU load-s...
18	B	`Knowledge`	What causes 'Thermal Throttling' in high-performan...
19	B	`Knowledge`	How wide are Intel AVX2 vector registers (YMM0 to ...
20	B	`Knowledge`	What is 'Dynamic Voltage and Frequency Scaling' (D...
21	B	`Understand`	What is the typical memory hierarchy capacity and ...

Q#	Correct Answer	Cognitive Bloom Level	Core Concept Focus
22	B	`Knowledge`	What is 'Spatial Locality' in computer memory acce...
23	A	`Knowledge`	What is 'Temporal Locality' in software execution?...
24	B	`Understand`	What occurs during a CPU 'Translation Lookaside Bu...
25	B	`Analyze`	How do Linux 'Hugepages' (e.g., 2MB or 1GB pages i...
26	B	`Understand`	What is the function of CPU 'Hardware Data Prefetc...
27	B	`Knowledge`	In the MESI cache coherence protocol, what does th...
28	B	`Analyze`	Why does traversing a two-dimensional C/C++ array ...
29	B	`Apply`	What is peak theoretical memory bandwidth for a se...
30	B	`Understand`	What is the primary architectural trade-off of Hig...
31	B	`Understand`	In the Message Passing Interface (MPI), what is th...
32	B	`Knowledge`	In an MPI communicator of N processes, what does...
33	B	`Understand`	What is 'Bisection Bandwidth' in HPC network topol...
34	B	`Understand`	Why is a non-blocking Fat-Tree network topology wi...
35	B	`Analyze`	What does RDMA (Remote Direct Memory Access) over ...
36	B	`Understand`	What is the primary role of Linux Control Groups (...
37	B	`Understand`	In Slurm job scheduling, what does the 'Backfill S...
38	B	`Knowledge`	What is the function of the Slurm daemon 'slurmdbd...
39	B	`Knowledge`	What does a Slurm job state of 'NODE_FAIL' indicat...
40	B	`Analyze`	Why is 'process affinity' (core pinning) essential...
41	B	`Understand`	In distributed deep learning, what distinguishes '...
42	B	`Analyze`	Why does Asynchronous SGD often experience converg...
43	B	`Understand`	In the Ring-AllReduce algorithm, how many message ...
44	B	`Apply`	According to the Linear Scaling Rule, if a model t...
45	B	`Understand`	Why is 'Gradual Learning Rate Warmup' crucial when...
46	B	`Analyze`	What is the primary cause of 'GPU Compute Starvati...
47	B	`Understand`	How does 'Communication-Computation Overlapping' h...
48	B	`Understand`	Why should deep learning model checkpoints be save...
49	B	`Knowledge`	What is the memory reduction factor achieved by tr...
50	B	`Analyze`	What is a 'Distributed Straggler', and why does it...

🔬 Complete Questions with Detailed Technical Rationale

Question 01

What is the primary operational difference between 'Profiling' and 'Debugging' in software engineering?

- **(a)** Profiling finds syntax errors; debugging finds logic errors.
- **(b) [CORRECT]** Debugging investigates functional correctness and crashes; profiling measures dynamic resource consumption (execution time, memory bandwidth, cache misses) to identify performance bottlenecks.
- **(c)** Profiling only works on Windows; debugging only works on Linux.
- **(d)** Debugging requires recompiling the operating system kernel.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: Debugging isolates logic bugs and faulty outputs, whereas profiling characterizes the dynamic resource usage of correct software to guide performance optimization.

Question 02

What is the primary drawback of using source-code 'Instrumentation' (e.g., injecting manual timers around every function) compared to statistical sampling?

- **(a)** Instrumentation cannot be compiled by GCC.
- **(b) [CORRECT]** Probe effect / overhead: inserting instrumentation code alters execution timing, pollutes instruction caches, and distorts the true performance characteristics of the application.
- **(c)** Instrumentation makes the application run 100 times faster.
- **(d)** Instrumentation only records network bandwidth.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: Software instrumentation perturbs execution by adding instructions and memory writes, distorting call timings and cache residency (the probe effect). Statistical sampling minimizes this overhead.

Question 03

What does the metric 'Instructions Per Cycle' (IPC) signify when evaluating CPU application efficiency?

- **(a)** The number of Python files executed per second.
- **(b) [CORRECT]** The average number of machine instructions retired by the processor execution pipeline during each clock cycle.
- **(c)** The rotational speed of the cooling fans.
- **(d)** The total amount of memory allocated on the heap.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: IPC ($IPC = \text{Instructions}/\text{Cycles}$) measures microarchitectural efficiency. Modern superscalar CPUs can retire 4-6 instructions per cycle; low IPC (< 1.0) indicates heavy pipeline stalling.

Question 04

According to Amdahl's Law, what limits the maximum theoretical speedup of an application when adding an infinite number of parallel processor cores?

- **(a)** The maximum capacity of the power supply.
- **(b) [CORRECT]** The strictly sequential (serial) fraction of the algorithm that cannot be parallelized.
- **(c)** The physical length of the Ethernet network cables.
- **(d)** The clock frequency of the system real-time clock.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: Amdahl's Law ($S_{max} = 1/s$, where s is the serial fraction) proves that the sequential portion of code imposes an absolute ceiling on parallel speedup, regardless of core count.

Question 05

In software profiling, what is a performance 'Hotspot'?

- **(a)** A Wi-Fi access point in the server room.
- **(b) [CORRECT]** A localized function or code section that consumes a disproportionately large fraction of the total execution time or processor cycles.
- **(c)** A CPU core that has exceeded its thermal limit.
- **(d)** A variable that stores sensitive user passwords.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: In performance analysis, a hotspot is a small region of code (e.g., an inner loop or GEMM routine) responsible for the majority of execution runtime, offering the greatest return on optimization effort.

Question 06

In a profiler call tree, what is the difference between 'Self Time' and 'Inclusive Time' (Cumulative Time)?

- **(a)** Self time includes compiler time; inclusive time does not.
- **(b) [CORRECT]** Self time measures time spent solely within the instructions of that specific function; inclusive time includes self time plus the time spent in all subroutines called by that function.
- **(c)** Inclusive time is measured in seconds; self time is measured in bytes.
- **(d)** Self time only counts idle wait states.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: Self time isolates execution cycles spent directly in the function body, while inclusive time aggregates time spent in the function and all its children down the call stack.

Question 07

Why is statistical sampling profiling considered superior to full function-entry/exit tracing for production HPC workloads?

- **(a)** Statistical sampling produces gigabytes of text files per second.
- **(b) [CORRECT]** Statistical sampling introduces minimal runtime overhead (< 2%), does not require binary recompilation, and avoids massive trace file growth.
- **(c)** Statistical sampling only records the exit code of the program.
- **(d)** Tracing cannot record floating-point operations.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: Statistical sampling periodically inspects the instruction pointer without modifying binary code, adding negligible overhead (< 2%) compared to intrusive tracing which can slow code down by 10x-100x.

Question 08

What is the difference between 'CPU Time' and 'Wall-Clock Time' reported by performance profiling tools?

- **(a)** CPU time is measured by a physical clock on the wall; wall-clock time is measured by the CPU.
- **(b) [CORRECT]** CPU Time is the total active execution time spent by all processor cores executing the process's instructions; Wall-Clock Time is the actual real-world elapsed time between process start and completion.
- **(c)** Wall-clock time is always greater than CPU time on multi-core systems.
- **(d)** CPU time does not count time spent in user space.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: Wall-clock time measures elapsed real-world time. For a multi-threaded process running across 8 cores simultaneously for 10 seconds of wall time, the total CPU time would be approximately 80 seconds.

Question 09

What problem arises when profiling multi-threaded applications if thread synchronizations (such as mutex locks or barriers) are frequent?

- **(a)** The CPU frequency drops to zero.
- **(b) [CORRECT]** Threads spend significant wall-clock time in idle wait states (waiting for locks/barriers), which may look like low CPU utilization rather than an active computation hotspot.
- **(c)** The operating system automatically terminates all threads.
- **(d)** The profiler crashes with a syntax error.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: Synchronization overhead manifests as idle wait time (futex stalls, spin-wait locks). Profilers must distinguish between productive computational hotspots and lock contention stalls.

Question 10

Why must an executable be compiled with debug symbols (e.g., `-g` flag in GCC/ICX) when preparing for in-depth profiling?

- **(a)** Without `-g`, the executable cannot run on 64-bit systems.
- **(b) [CORRECT]** Debug symbols provide the mapping (DWARF/PDB tables) between machine instruction addresses and original source-code filenames and line numbers, enabling line-by-line profiling.
- **(c)** Debug symbols automatically disable all compiler optimizations.
- **(d)** Debug symbols allow the compiler to send data over the internet.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: The `-g` flag embeds DWARF symbol tables into the ELF binary, allowing profilers to map sampled instruction pointers back to exact source code lines and function names.

Question 11

In modern superscalar processors, what is the role of the 'Out-of-Order Execution' (OoO) engine?

- **(a)** Executing instructions in random order to scramble memory for security.
- **(b) [CORRECT]** Executing instructions as soon as their data dependencies are ready and execution units are free, rather than waiting in strict program order, hiding memory access delays.
- **(c)** Reversing the flow of electrical current through the silicon gates.
- **(d)** Running instructions backwards to implement undo operations.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: Out-of-order execution allows independent instructions to execute ahead of time while earlier instructions are stalled waiting for data from cache or RAM, maximizing instruction-level parallelism.

Question 12

What happens when a CPU branch predictor incorrectly predicts the outcome of a conditional branch instruction?

- **(a)** The processor permanently halts and requires a hardware reboot.
- **(b) [CORRECT]** Branch Misprediction Penalty: all speculatively executed uOps must be discarded (flushed from the pipeline), and instruction fetching must restart from the correct target address, wasting 15-20+ clock cycles.
- **(c)** The program automatically branches to line 0 of the code.
- **(d)** The operating system switches to 16-bit real mode.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: A misprediction forces a pipeline flush, discarding all speculatively decoded and executed work in the reorder buffer and incurring a 15-20+ cycle penalty to refill the pipeline.

Question 13

What is the purpose of the 'Reorder Buffer' (ROB) in an out-of-order processor architecture?

- **(a)** Sorting disk sectors for hard drive defragmentation.

- **(b) [CORRECT]** Ensuring that instructions that executed out-of-order commit their architectural register and memory modifications in strict original program order, preserving precise exceptions.
- **(c)** Reordering print statements in Python programs.
- **(d)** Compressing executable files in system RAM.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: The Reorder Buffer (ROB) holds in-flight instructions and guarantees that architectural state changes commit in-order, ensuring that exceptions, interrupts, and program logic remain strictly deterministic.

Question 14

What is 'Register Renaming' in superscalar processors, and what performance hazard does it eliminate?

- **(a)** Changing variable names in source code to make them shorter.
- **(b) [CORRECT]** Mapping architectural registers to a larger pool of physical registers to eliminate false dependencies: Write-After-Read (WAR) and Write-After-Write (WAW) hazards.
- **(c)** Renaming thread identifiers in the Linux scheduler.
- **(d)** Converting floating point numbers to integers.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: Register renaming dynamically maps limited architectural register names to hundreds of internal physical registers, breaking false dependencies caused by register reuse without true data flow.

Question 15

What does a Fused Multiply-Add (FMA) hardware instruction compute in a single clock cycle?

- **(a)** $d = a + b + c$
- **(b) [CORRECT]** $d = (a \times b) + c$
- **(c)** $d = (a / b) - c$
- **(d)** $d = \sqrt{a^2 + b^2}$

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: FMA performs the operation $d = (a \times b) + c$ with a single rounding step, doubling floating-point throughput compared to separate multiply and add instructions and improving precision.

Question 16

What is 'Simultaneous Multi-Threading' (SMT), known commercially as Intel Hyper-Threading?

- **(a)** Placing two distinct CPU motherboards inside a single chassis.
- **(b) [CORRECT]** Exposing two logical execution contexts (architectural register states) per physical core, sharing execution units and caches to interleave execution during memory stalls.
- **(c)** Running two copies of the operating system simultaneously.

- **(d)** Executing code on both the CPU and GPU at the same time.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: SMT duplicates architectural state (registers, program counter) within a single physical core, allowing the scheduler to feed execution units from a second thread when the first thread stalls on memory.

Question 17

What is 'Store-to-Load Forwarding' in a CPU load-store execution unit?

- **(a)** Uploading files to cloud storage before reading them.
- **(b) [CORRECT]** Forwarding data directly from an active store buffer to a subsequent load instruction requesting the same memory address, bypassing cache and DRAM access entirely.
- **(c)** Converting memory stores into network packets.
- **(d)** Storing CPU registers to the stack on function calls.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: When a load instruction needs data recently written by a preceding store that has not yet committed to L1 cache, store forwarding passes the byte directly from the store buffer to the load register.

Question 18

What causes 'Thermal Throttling' in high-performance computing processors?

- **(a)** The processor temperature drops below 0 degrees Celsius.
- **(b) [CORRECT]** The silicon die temperature reaches the maximum junction threshold ($T_j\max$), triggering hardware thermal watchdogs that dynamically downclock CPU frequency to prevent physical damage.
- **(c)** The operating system runs out of disk storage.
- **(d)** The liquid cooling tubes become transparent.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: When silicon reaches junction limit (~100°C), hardware PROCHOT signals trigger DVFS circuits to reduce clock frequency and operating voltage, preventing irreversible semiconductor degradation.

Question 19

How wide are Intel AVX2 vector registers (YMM0 to YMM15), and how many 64-bit double-precision floats can one register hold?

- **(a)** 128 bits wide, holding 2 doubles.
- **(b) [CORRECT]** 256 bits wide, holding 4 double-precision (FP64) floats.
- **(c)** 512 bits wide, holding 8 doubles.
- **(d)** 64 bits wide, holding 1 double.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: AVX2 registers (YMM) are 256 bits wide. Since an IEEE 754 double-precision float is 64 bits, $256/64 = 4$ double-precision floats can be processed simultaneously per vector instruction.

Question 20

What is 'Dynamic Voltage and Frequency Scaling' (DVFS) in modern microprocessors?

- **(a)** A technique to convert alternating current (AC) to direct current (DC).
- **(b) [CORRECT]** A hardware power management feature that dynamically adjusts operating clock frequency and core supply voltage based on computational workload demands and thermal limits.
- **(c)** A software algorithm for resizing JPEG images.
- **(d)** A compiler optimization for unrolling loops.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: DVFS dynamically scales voltage and frequency (e.g., P-states, Intel Speed Shift) to optimize power efficiency and maximize Turbo Boost frequency within TDP envelopes.

Question 21

What is the typical memory hierarchy capacity and latency progression on modern Intel Xeon Scalable processors?

- **(a)** L1 is slowest and largest; DRAM is fastest and smallest.
- **(b) [CORRECT]** L1 (32-48 KB, ~1 ns / 4 cycles) -> L2 (1-2 MB, ~3-4 ns / 14 cycles) -> L3/LLC (shared ~1.5-2.5 MB/core, ~12-20 ns / 40-50 cycles) -> DRAM (~60-90 ns / 200+ cycles).
- **(c)** All cache levels have identical capacity and latency.
- **(d)** DRAM is on-chip; L1 cache is on an external PCIe card.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: Memory hierarchy trades capacity for latency: small, ultra-fast L1/L2 caches sit directly adjacent to core execution units, backed by large shared L3 LLC and high-capacity off-chip DRAM.

Question 22

What is 'Spatial Locality' in computer memory access patterns?

- **(a)** Accessing the exact same memory address repeatedly over time.
- **(b) [CORRECT]** The principle that if a particular memory location is accessed, nearby contiguous memory locations are likely to be accessed soon thereafter.
- **(c)** Storing data on satellites in space.
- **(d)** Distributing files across multiple hard drives.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: Spatial locality means adjacent memory addresses are referenced in sequence (e.g., iterating through a contiguous C array), which exploits the 64-byte cache line fetched by hardware prefetchers.

Question 23

What is 'Temporal Locality' in software execution?

- **(a) [CORRECT]** The principle that data or instructions accessed recently are likely to be accessed again in the near future.
- **(b)** Accessing data only on weekends.
- **(c)** Measuring memory transfer speeds using a stopwatch.
- **(d)** Converting memory addresses into timestamps.

> **Correct Answer**: **(A)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: Temporal locality means data accessed once is reused soon after (e.g., accumulating into a loop accumulator variable), which benefits from remaining resident in L1 cache.

Question 24

What occurs during a CPU 'Translation Lookaside Buffer' (TLB) miss?

- **(a)** The computer converts English text to Spanish.
- **(b) [CORRECT]** The hardware MMU must perform a high-latency 'page table walk' across multi-level page tables in main memory to resolve the virtual-to-physical address translation.
- **(c)** The CPU registers are wiped to zero.
- **(d)** The process terminates immediately with an error.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: The TLB caches recent virtual-to-physical page mappings. A TLB miss forces the Memory Management Unit (MMU) to traverse 4-level or 5-level page tables in memory, stalling the core for 30-100 cycles.

Question 25

How do Linux 'Hugepages' (e.g., 2MB or 1GB pages instead of default 4KB pages) benefit high-throughput HPC workloads?

- **(a)** They allow text files to have larger font sizes.
- **(b) [CORRECT]** They drastically increase the total physical memory range covered by a fixed number of TLB entries, slashing TLB misses by orders of magnitude for large data structures.
- **(c)** They eliminate the need for system RAM.
- **(d)** They prevent disk drives from filling up.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: Default 4KB pages require thousands of TLB entries to cover large matrices or model weights. 2MB hugepages cover 512x more address space per entry, keeping virtual memory translation overhead near zero.

Question 26

What is the function of CPU 'Hardware Data Prefetchers' (e.g., Stream Prefetcher)?

- **(a)** Downloading software updates automatically from the internet.
- **(b) [CORRECT]** Detecting sequential or strided memory access patterns and speculatively loading anticipated cache lines from DRAM into L2/L1 cache before the CPU execution core explicitly requests them.
- **(c)** Writing random numbers into empty RAM sectors.
- **(d)** Checking for syntax errors in Python scripts.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: Hardware stream prefetchers observe memory address deltas. Upon detecting regular stride patterns, they proactively pull upcoming cache lines into L2/L1 to hide memory latency.

Question 27

In the MESI cache coherence protocol, what does the 'I' (Invalid) state indicate?

- **(a)** The processor is idle.
- **(b) [CORRECT]** The cache line does not contain valid data, meaning any read or write to this address by the core will result in a cache miss requiring a fetch from LLC or DRAM.
- **(c)** The memory address is corrupt.
- **(d)** The cache line is exclusively owned and modified.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: In MESI (Modified, Exclusive, Shared, Invalid), the Invalid state indicates the cache line has been invalidated (e.g., by another core writing to the address), requiring a reload upon next access.

Question 28

Why does traversing a two-dimensional C/C++ array column-by-column (strided) perform significantly slower than row-by-row?

- **(a)** Because C/C++ uses column-major order like Fortran.
- **(b) [CORRECT]** Because C/C++ stores 2D arrays in row-major order; column-by-column traversal jumps by N elements between consecutive accesses, causing a cache miss on virtually every element and defeating prefetchers.
- **(c)** Because compiler optimization flags only apply to rows.
- **(d)** Because column traversal disables CPU vector instructions.

Correct Answer: **(B)**
Bloom's Cognitive Level: `Analyze`
Architectural Rationale: In row-major order, `arr[i][j]` and `arr[i][j+1]` are adjacent in memory. Traversing `arr[i][j]` followed by `arr[i+1][j]` jumps by row size ($N \times \text{sizeof}$), blowing past the 64-byte cache line and thrashing caches.

Question 29

What is peak theoretical memory bandwidth for a server featuring an 8-channel DDR5-4800 memory subsystem?

- **(a)** 38.4 GB/s
- **(b) [CORRECT]** 307.2 GB/s
- **(c)** 1000 GB/s
- **(d)** 4.8 GB/s

Correct Answer: **(B)**
Bloom's Cognitive Level: `Apply`
Architectural Rationale: DDR5-4800 transfers 4.8 billion transfers/sec per pin. Each channel is 64 bits (8 bytes): $4.8\ \text{GT/s} \times 8\ \text{bytes} = 38.4\ \text{GB/s}$ per channel. Across 8 channels: $8 \times 38.4 = 307.2\ \text{GB/s}$.

Question 30

What is the primary architectural trade-off of High Bandwidth Memory (HBM) compared to standard DDR5 DIMMs?

- **(a)** HBM is slower but much cheaper.
- **(b) [CORRECT]** HBM delivers massively higher bandwidth (2-3+ TB/s vs ~300 GB/s) and energy efficiency via thousands of wide interposer connections, but has lower maximum capacity and cannot be upgraded via motherboard slots.
- **(c)** HBM cannot store floating-point numbers.
- **(d)** HBM requires liquid nitrogen cooling at all times.

Correct Answer: **(B)**
Bloom's Cognitive Level: `Understand`
Architectural Rationale: HBM integrates 3D-stacked DRAM dies directly on-package via a silicon interposer, delivering multi-terabyte/sec bandwidth, but is fixed in capacity (e.g., 64-96GB) and cannot be expanded like modular DDR DIMMs.

Question 31

In the Message Passing Interface (MPI), what is the difference between 'MPI_Send' (blocking) and 'MPI_Isend' (non-blocking)?

- **(a)** MPI_Send sends numbers; MPI_Isend sends text.
- **(b) [CORRECT]** MPI_Send blocks the calling process until the send buffer can be safely modified; MPI_Isend returns immediately and returns a request handle, allowing computation to overlap with message transit.
- **(c)** MPI_Isend is only available on single-core laptops.
- **(d)** MPI_Send requires root privileges.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: Non-blocking communication (`MPI_Isend`, `MPI_Irecv`) initiates the transfer and returns immediately, allowing the application to execute useful computation while the network adapter transfers the payload in the background.

Question 32

In an MPI communicator of N processes, what does 'MPI_Allgather' perform?

- **(a)** It sums all numbers and leaves the result only on rank 0.
- **(b) [CORRECT]** Each process contributes a chunk of data, and all chunks are concatenated such that every process in the communicator receives the complete combined dataset.
- **(c)** It deletes all processes except rank 0.
- **(d)** It broadcasts an identical message from root to all.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: MPI_Allgather collects equal data blocks from all N processes and delivers the complete gathered buffer of size $N \times M$ to every participating process.

Question 33

What is 'Bisection Bandwidth' in HPC network topologies?

- **(a)** The maximum bandwidth of a single USB port.
- **(b) [CORRECT]** The minimum data transfer capacity between two equal halves of the network when the network is partitioned in the worst possible way.
- **(c)** The speed of light in optical fiber.
- **(d)** The frequency of the cluster master clock.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: Bisection bandwidth measures network capacity across the narrowest cross-section dividing the cluster in half. High bisection bandwidth (e.g., non-blocking fat-tree) guarantees full speed for all-to-all communication.

Question 34

Why is a non-blocking Fat-Tree network topology widely deployed in HPC and AI supercomputers?

- **(a)** It looks like a pine tree in datacenter blueprints.
- **(b) [CORRECT]** It provides full bisection bandwidth, guaranteeing that any node can communicate with any other node at full link line-rate simultaneously without cross-switch oversubscription.
- **(c)** It requires only 1 network switch for 10,000 servers.
- **(d)** It operates without physical cabling.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: A non-blocking fat-tree increases total bandwidth toward the root, ensuring that network links never become oversubscribed even when all nodes engage in simultaneous All-to-All or AllReduce traffic.

Question 35

What does RDMA (Remote Direct Memory Access) over Converged Ethernet (RoCE v2) eliminate compared to standard TCP/IP networking?

- **(a)** It eliminates the need for Ethernet switches.
- **(b) [CORRECT]** It eliminates operating system kernel involvement, intermediate socket buffer copies, and CPU interrupt handling, performing zero-copy memory transfers directly across network adapters.
- **(c)** It eliminates physical Ethernet MAC addresses.
- **(d)** It replaces fiber-optic cables with copper wires.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: Standard TCP/IP copies packets between user space and kernel space buffers, triggering CPU interrupts. RoCE v2 uses hardware NIC state machines to read/write remote RAM directly, achieving microsecond latencies.

Question 36

What is the primary role of Linux Control Groups (cgroups) in cluster workload management like Slurm?

- **(a)** Encrypting student source code.
- **(b) [CORRECT]** Enforcing strict hardware resource limits (constraining CPU cores, physical RAM, and GPU devices) to prevent user jobs from interfering with each other or crashing host nodes.
- **(c)** Managing user graphical user interfaces.
- **(d)** Converting shell scripts to C++ code.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: Slurm uses cgroups (e.g., `cpuset`, `memory`) to isolate jobs, restricting processes to allocated cores and memory budgets, instantly terminating processes that exceed limits (OOM kills) before they destabilize the node.

Question 37

In Slurm job scheduling, what does the 'Backfill Scheduler' accomplish?

- **(a)** Deleting old completed jobs from the database.
- **(b) [CORRECT]** Starting smaller, lower-priority jobs on idle nodes ahead of time, provided they will complete before higher-priority pending jobs are scheduled to begin, maximizing cluster utilization.
- **(c)** Re-running failed jobs with lower memory requests.
- **(d)** Backing up node hard drives to magnetic tape.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: Backfill scheduling inspects the queue to find short jobs that can fit into idle resource windows without delaying the projected start time of high-priority jobs, keeping node utilization high.

Question 38

What is the function of the Slurm daemon 'slurmdbd'?

- **(a)** Running Python machine learning models on GPU nodes.
- **(b) [CORRECT]** Managing the persistent Slurm database (MySQL/MariaDB) to store accounting records, fair-share usage history, user allocations, and job completion telemetry.
- **(c)** Monitoring CPU temperature across servers.
- **(d)** Serving web pages for cluster documentation.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: `slurmdbd` is the Slurm Database Daemon that interfaces with MySQL/MariaDB, recording historical job runtime, CPU-hours consumed, and fair-share priorities for cluster governance.

Question 39

What does a Slurm job state of 'NODE_FAIL' indicate?

- **(a)** The user entered an invalid password.
- **(b) [CORRECT]** The job was terminated because the physical compute node hosting the job experienced a hardware fault (e.g., kernel panic, power failure, or hardware watchdog reset).
- **(c)** The Python interpreter found a syntax error.
- **(d)** The model reached maximum training accuracy.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: `NODE_FAIL` indicates that the Slurm controller lost heartbeat communication with the compute node's `slurmd` daemon due to a kernel crash, hardware fault, or unrecoverable system freeze.

Question 40

Why is 'process affinity' (core pinning) essential for MPI applications running on modern NUMA supercomputers?

- **(a)** To make processes run in alphabetical order.
- **(b) [CORRECT]** To prevent the OS kernel scheduler from migrating threads across CPU sockets, which destroys L1/L2 cache residency and forces high-latency remote NUMA memory traffic.
- **(c)** To disable Intel Turbo Boost.
- **(d)** To ensure processes can only read read-only files.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: Without core affinity, the Linux scheduler frequently migrates active threads across cores or sockets to balance load, invalidating hot L1/L2 caches and causing catastrophic performance degradation.

Question 41

In distributed deep learning, what distinguishes 'Synchronous Data Parallelism' from 'Asynchronous Data Parallelism'?

- **(a)** Synchronous uses GPUs; asynchronous uses CPUs.
- **(b) [CORRECT]** Synchronous waits for all workers to compute and average gradients before updating weights at a synchronization barrier; asynchronous allows workers to update weights independently without waiting.
- **(c)** Asynchronous training guarantees bit-exact mathematical reproducibility.
- **(d)** Synchronous training cannot scale beyond 2 workers.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: Synchronous training enforces an AllReduce barrier where all workers synchronize gradients before updating parameters, ensuring mathematically consistent updates at the cost of waiting for the slowest node.

Question 42

Why does Asynchronous SGD often experience convergence degradation and lower final model accuracy at large scale?

- **(a)** Because workers forget their learning rates.
- **(b) [CORRECT]** The Stale Gradient Problem: by the time a worker finishes calculating gradients, other workers have already updated the shared weights, causing updates to be applied to an outdated parameter state.
- **(c)** Because asynchronous training uses 8-bit integers.
- **(d)** Because Python does not support multi-threading.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: In async training, workers compute gradients against model state W_t. By the time gradients arrive, the model has advanced to W_{t+k}. Applying stale gradients destabilizes optimization and hurts convergence.

Question 43

In the Ring-AllReduce algorithm, how many message transfer steps are required to synchronize N workers during the Scatter-Reduce phase?

- **(a)** 1 step
- **(b) [CORRECT]** $N - 1$ steps
- **(c)** N^2 steps
- **(d)** 2^N steps

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: The Scatter-Reduce phase divides the tensor into N equal chunks. At each step, workers send one chunk to their neighbor and receive one chunk. After $N-1$ steps, each worker holds the finalized sum of one chunk.

Question 44

According to the Linear Scaling Rule, if a model trained on 1 GPU with batch size 32 has a base learning rate of 0.001, what should the learning rate be when scaling to 16 GPUs with global batch size 512?

- **(a)** 0.0000625
- **(b) [CORRECT]** 0.016
- **(c)** 0.001
- **(d)** 0.512

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Apply`
> **Architectural Rationale**: Under the Linear Scaling Rule: $LR_{new} = LR_{base} \times N = 0.001 \times 16 = 0.016$. Scaling the learning rate proportionally keeps the expectation of weight updates per step consistent with the expanded batch.

Question 45

Why is 'Gradual Learning Rate Warmup' crucial when applying large learning rates derived from the Linear Scaling Rule?

- **(a)** To allow the cooling fans to spin up slowly.
- **(b) [CORRECT]** Because in early training iterations, randomly initialized weights produce large, volatile gradients; applying a massive learning rate immediately causes loss explosion and divergent optimization.
- **(c)** Because Slurm kills jobs that start with high learning rates.
- **(d)** To allow the dataset to be downloaded from the network.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: In early iterations, model parameters are random and gradients are noisy. Ramping the learning rate linearly from zero to the target rate over the first 5 epochs prevents catastrophic gradient explosion.

Question 46

What is the primary cause of 'GPU Compute Starvation' in deep learning training clusters?

- **(a)** The GPU lacks floating-point registers.
- **(b) [CORRECT]** The host CPU data loading pipeline (image decoding, data augmentation, batch shuffling) cannot prepare batches fast enough to keep GPU tensor cores continuously supplied.
- **(c)** The model has too many parameters.
- **(d)** The learning rate is set too low.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: If CPU data loading and PCIe transfer throughput cannot match accelerator compute consumption, GPUs spend significant time idle waiting for batches, visible as low GPU compute utilization.

Question 47

How does 'Communication-Computation Overlapping' hide network latency in distributed deep learning frameworks?

- **(a)** By executing training without using a network.
- **(b) [CORRECT]** By launching the asynchronous AllReduce gradient synchronization for outer layers as soon as their gradients are calculated, overlapping network transfer with the backward pass of earlier layers.
- **(c)** By running data loading on the GPU.
- **(d)** By skipping the forward pass on every second batch.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: Because backpropagation calculates gradients sequentially from layer L to layer 1, the framework immediately dispatches AllReduce for layer L over the network while the accelerator computes layer $L-1$, hiding network latency.

Question 48

Why should deep learning model checkpoints be saved strictly by rank 0 (`if hvd.rank() == 0`) in distributed cluster environments?

- **(a)** Only rank 0 has access to model weights.
- **(b) [CORRECT]** If all distributed workers write checkpoints simultaneously to the same shared filesystem path, severe file locking conflicts, race conditions, and filesystem corruption will occur.
- **(c)** Rank 0 is the only node with internet access.
- **(d)** Other ranks will delete the model checkpoint.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: Because all workers maintain synchronized weights after every step, writing the model checkpoint from multiple ranks simultaneously onto a shared parallel filesystem (NFS/Lustre) creates write collisions, file corruption, and massive I/O serialization.

Question 49

What is the memory reduction factor achieved by transitioning model weights and activations from FP32 (32-bit float) to BF16 (16-bit Bfloat)?

- **(a)** $1.5\times$
- **(b) [CORRECT]** $2\times$ (50% reduction in memory footprint)
- **(c)** $4\times$
- **(d)** $8\times$

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: FP32 requires 4 bytes per parameter, while BF16 requires 2 bytes ($16/8 = 2$). This cuts parameter and activation memory consumption exactly in half ($2\times$ reduction).

Question 50

What is a 'Distributed Straggler', and why does it disproportionately degrade synchronous distributed training performance?

- **(a)** A worker that runs faster than all other workers.
- **(b) [CORRECT]** A single slow worker node (due to hardware throttling, network contention, or I/O stalls) that forces all other worker nodes to stall at the synchronization barrier until it catches up.
- **(c)** A network packet that gets lost.
- **(d)** A dataset sample that contains corrupted labels.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: Because synchronous distributed training requires all ranks to participate in the AllReduce barrier before taking an optimizer step, the step time of the entire 100-node cluster is dictated by the slowest single node.
